

Cattle IoT — Technical Document

This document captures **development-focused details** for the ESP32 cattle collar, backend, and Godhan app integration.

1. Firmware (ESP32)

- **Core features:**
 - Sensor sampling (IMU, temperature, rumination mic).
 - Local aggregation and averaging.
 - MQTT publish every 5 min.
 - Deep-sleep between uploads.
 - Fail-safe local storage (SPIFFS).
 - OTA firmware updates (HTTP/MQTT triggered).
 - **Config methods:**
 - BLE provisioning (Wi-Fi SSID, password, cattle_id).
 - Captive portal fallback.
 - **Security:**
 - Device ID + secret key provisioned.
 - TLS (MQTTS) communication.
 - Firmware signature validation before flashing.
-

2. Backend Services

- **Message Broker:** MQTT (Mosquitto/EMQX).
 - **Data Ingestion:** MQTT → Node.js service → MongoDB/Influx.
 - **Analytics:**
 - Estrus detection (activity ↑, rumination ↓).
 - Calving prediction (gestation window + signal anomalies).
 - **APIs:** REST/GraphQL for app integration.
-

3. Database Models (Mongoose ODM)

Cattle

```
const CattleSchema = new mongoose.Schema({
  cattle_id: { type: String, unique: true },
  farm_id: String,
  breed: String,
  tag_number: String,
```

```

    dob: Date,
    lactation_stage: Number,
    insemination_date: Date,
    pregnancy_confirmed: { status: Boolean, date: Date },
    expected_calving_date: Date,
    actual_calving_date: Date,
    assigned_device: { type: String, ref: 'Device' },
    created_at: { type: Date, default: Date.now }
  });

```

Device

```

const DeviceSchema = new mongoose.Schema({
  device_id: { type: String, unique: true },
  cattle_id: { type: String, ref: 'Cattle' },
  farm_id: String,
  fw_version: String,
  last_seen: Date,
  battery: Number,
  status: { type: String, enum: ['active', 'offline', 'needs_update'] },
  assigned_at: Date
});

```

Sensor Data

```

const SensorDataSchema = new mongoose.Schema({
  device_id: String,
  cattle_id: String,
  timestamp: { type: Date, default: Date.now },
  temp: Number,
  accX: Number,
  accY: Number,
  accZ: Number,
  rumination_events: Number,
  battery: Number,
  status_flags: [String]
});

```

Alerts

```

const AlertSchema = new mongoose.Schema({
  alert_id: { type: String, unique: true },
  device_id: String,
  cattle_id: String,

```

```

    farm_id: String,
    timestamp: { type: Date, default: Date.now },
    type: { type: String, enum:
[ 'HEAT_DETECTED', 'CALVING', 'FEVER', 'LOW_BATTERY', 'DEVICE_OFFLINE' ] },
    severity: { type: String, enum: [ 'info', 'warning', 'critical' ] },
    message: String
  });

```

Firmware

```

const FirmwareSchema = new mongoose.Schema({
  fw_version: { type: String, unique: true },
  release_date: Date,
  url: String,
  notes: String,
  status: { type: String, enum: [ 'latest', 'deprecated' ] }
});

```

4. APIs

Device Management

- POST /api/devices/register
- POST /api/devices/{id}/assign
- GET /api/devices/{id}

Data

- POST /api/data (MQTT → REST bridge optional)
- GET /api/cattle/{cattle_id}/data
- GET /api/cattle/{cattle_id}/alerts

Firmware

- POST /api/devices/{id}/update
- GET /api/firmware/latest

Events

- POST /api/cattle/{id}/insemination
- POST /api/cattle/{id}/pregnancy
- POST /api/cattle/{id}/calving

5. Analytics Modules

Heat Detection

- **Inputs:** activity $\uparrow$, rumination $\downarrow$, temp changes.
- **Rule-based (MVP):** activity > baseline + 2σ AND rumination < 80% baseline.
- **ML Extension:** RandomForest/XGBoost on window features.
- **Output:** estrus probability + best insemination window.

Calving Prediction

- **Inputs:** insemination date, pregnancy confirmation, temp/rumination/activity signals.
 - **Gestation-based:** expected_calving_date = insemination + 283d.
 - **Signal refinement:** temp drop $\sim 0.5^\circ\text{C}$, rumination $\downarrow > 30\%$, restlessness $\uparrow$.
 - **Alerts:** 14d, 7d, 3d, imminent calving.
-

6. Security

- Device-level TLS + HMAC signing.
 - JWT auth for app/backend.
 - OTA firmware signature validation.
 - Data encrypted at rest.
-

7. Deployment

- **Farm Edge Setup:** Wi-Fi APs in barn $\rightarrow$ local MQTT broker $\rightarrow$ sync with cloud.
 - **Cloud Stack:** Dockerized Node.js API, MongoDB/InfluxDB, EMQX broker.
 - **Monitoring:** Grafana dashboards, device heartbeat monitoring.
-

8. Roadmap (Tech)

1. Pilot: 10–20 devices, Wi-Fi, rule-based estrus alerts.
 2. Add insemination/pregnancy logging in backend + app.
 3. Calving prediction logic (gestation + signal rules).
 4. Collect labels $\rightarrow$ train ML models (estrus, calving).
 5. Scale with OTA updates, edge compute, offline mode.
-